AssetIQ: Smart Asset Management Platform

Software Design Document (Deliverable 2)

Project: AssetIQ

Target Architecture: MERN Stack + Docker Containerization

Document Purpose: System architecture, database schematics, and design rationale.

1. Problem Understanding

Organizations, campuses, and production teams frequently struggle to manage shared physical resources (e.g., cameras, lighting, costumes). The reliance on manual tracking, spreadsheets, or verbal agreements leads to critical inefficiencies:

- **Double-Bookings:** Assets promised to multiple users simultaneously.
- **Inventory Shrinkage:** Lack of accountability for unreturned or delayed items.
- **Workflow Friction:** No standardized method for users to discover available assets or for administrators to approve requests.

The Solution: AssetIQ is a centralized, Role-Based Access Control (RBAC) platform that digitalizes asset discovery, automates the request-approval state machine, and dynamically tracks inventory levels in real time to prevent overallocation.

2. System Architecture

AssetIQ utilizes a containerized **MERN Stack** (MongoDB, Express.js, React, Node.js) architecture. The system is decoupled into independent layers, connected via RESTful API paradigms, and orchestrated using Docker Compose for frictionless, environment-agnostic deployment.

Architecture Layers:

1. **Presentation Layer (Frontend):** * **Tech:** React 18, Vite, Tailwind CSS, Recharts.
    - **Role:** Single Page Application (SPA) handling UI rendering, local state management, form validation, and data visualization. Communicates with the backend via Axios.
2. **Application Layer (Backend API):**
    - **Tech:** Node.js, Express.js, JSON Web Tokens (JWT).

- **Role:** Serves as the central logic hub. Intercepts HTTP requests, validates JWT authorization tokens, enforces role-based permissions (admin vs consumer), and executes business logic (e.g., deducting inventory upon booking approval).

3. **Data Layer (Database):**
    - **Tech:** MongoDB, Mongoose ORM.
    - **Role:** NoSQL document storage providing highly scalable and flexible data persistence for Users, Assets, and Bookings.
4. **DevOps Layer (Orchestration):**
    - **Tech:** Docker, Docker Compose.
    - **Role:** Isolates the frontend, backend, and database into unified, reproducible containers connected via a secure internal Docker network.

3. Database Schema

The database leverages MongoDB's document-based structure, strictly modeled using Mongoose schemas to ensure data integrity.

3.1 User Collection

Field	Type	Attributes	Description
_id	ObjectId	Primary Key, Auto	Unique identifier.
name	String	Required	Full name of the user.
email	String	Required, Unique	Login identifier.
password	String	Required	Bcrypt hashed password string.
role	String	Enum, Default:'consumer'	Access level (admin or consumer).

3.2 Asset Collection

Field	Type	Attributes	Description
_id	ObjectId	Primary Key, Auto	Unique identifier.
name	String	Required	Display name of the item.
category	String	Required	Dynamic grouping (e.g., Audio, Lighting).
description	String	Required	Details or specifications of the item.
quantity	Number	Required, Min: 0	Current available stock.
status	String	Enum	Available, Low Stock, Out of Stock.

3.3 Booking Collection

Field	Type	Attributes	Description
_id	ObjectId	Primary Key, Auto	Unique identifier.
user	ObjectId	Ref: 'User'	The consumer requesting the asset.
asset	ObjectId	Ref: 'Asset'	The specific asset being requested.
quantity	Number	Required, Min: 1	Number of items requested.
startDate	Date	Required	Expected checkout date.

Field	Type	Attributes	Description
endDate	Date	Required	Expected return date.
purpose	String	Required	Justification for the request.
status	String	Enum, Default:'Pending'	Pending, Approved, Rejected, Returned.
actualReturnDate	Date	Optional	Timestamp when item was physically returned.

4. Entity Relationship Diagram (ERD)

The database utilizes referencing (normalization) rather than embedding, ensuring that updates to an Asset's details immediately reflect across all associated Bookings.

Relationships:

- **User to Booking (1 : M):** A single user can generate multiple booking requests over time.
- **Asset to Booking (1 : M):** A single asset type can be associated with multiple separate booking requests from different users.

Data Flow map:

[USER] (1) ----------> (Many) **[BOOKING]** (Many) <---------- (1) **[ASSET]**

Primary Key: _id ..... Primary Key: _id ..... Primary Key: _id

Foreign Key: user_id ..... Foreign Key: asset_id ***

5. API Overview

The backend exposes a RESTful API protected by custom middleware (protect for authentication, admin for authorization).

Authentication Routes

- **POST /api/auth/register** (Public) - Registers a new user.
- **POST /api/auth/login** (Public) - Authenticates credentials and returns a JWT.

Asset Management Routes

- **GET /api/assets** (Protected) - Fetches all assets. Includes query params for searching.
- **POST /api/assets** (Admin Only) - Creates a new asset.
- **PUT /api/assets/:id** (Admin Only) - Updates an existing asset's details or quantity.
- **DELETE /api/assets/:id** (Admin Only) - Removes an asset from the system.

Booking & Workflow Routes

- **POST /api/bookings** (Protected) - Creates a new booking request. Verifies requested quantity does not exceed available inventory.
- **GET /api/bookings/mybookings** (Protected) - Fetches historical/active bookings specific to the logged-in user.
- **GET /api/bookings** (Admin Only) - Fetches all system-wide bookings for the Approval Dashboard.
- **PUT /api/bookings/:id/status** (Admin Only) - Transitions booking state. Executes inventory deduction logic upon Approved and inventory restoration upon Returned.

6. Design Decisions & Rationale

1. MongoDB & Mongoose over SQL

Rationale: Asset tracking inherently involves variable data attributes. A NoSQL approach provides flexibility for future feature expansion (e.g., adding dynamic metadata fields for specific asset types). Mongoose enforces strict schema validation at the application level, offering the best of both flexibility and integrity.

2. Just-In-Time Inventory Deduction

Rationale: Inventory quantities are *not* deducted when a user creates a Pending request. Deductions only occur when an Admin transitions the state to Approved. This prevents bad actors or abandoned requests from artificially locking up campus resources.

3. Stateless Authentication (JWT)

Rationale: Instead of using server-side sessions, the system utilizes JSON Web Tokens stored in HTTP headers. This aligns with RESTful principles, allows the backend to remain entirely stateless, and reduces database read overhead during route protection checks.

4. Dockerized Deployment

Rationale: To fulfill the "1-Click Deployment" requirement, Docker Compose is utilized to orchestrate the Node.js API, Vite Frontend, and MongoDB database. This eliminates environment discrepancies ("works on my machine" syndrome) and allows evaluators to launch the entire platform without managing local dependencies.

5. Frontend State & UI Choices

Rationale: React was selected for its component-based reusability. Tailwind CSS was utilized to accelerate UI development and ensure a responsive, modern interface. Recharts was implemented to provide administrators with immediate visual context regarding system utilization without requiring heavy third-party dashboard integrations.